

Analysis on procedural game and level design

Procedurality is the method of creating “[artifacts](#)” (maps, names, stories, quests) via algorithm rather than by hand. Giving the engine a series of rules, it can create content on its own.

It is often used to create a huge number of artifacts in a relatively small amount of time, or to allow the developers, or the players, to create content indefinitely.

Every generator is **linked** to the needs of the developers, and created to satisfy such necessities.



Guidance

The generator needs properties to create bad and good artifacts. The easiest generators are the ones where you can describe “**good**” artifacts and set concrete **properties**. Define the constraints wanted and not in the artifacts.

The range defined by the constraints determine how wide is the possibility for the generator to create something (possibly) good.

The **iteration between identity and quantity** is always needed, for more constraints will reduce the differences between the artifacts. Less constraints, to possibly give every artifact his own identity, might end in creating similar results.

This topic will be deepened later in the document.



[Differentiation of the same urban level, XCOM2](#)

Methods

- Encapsulate knowledge of options (A)
- Create some structure (B)
- Encode conditional rules for options (A2)
- Create variability in structure (B2)
- Be able to ask itself questions about its constraints (have I solved this?) (C)

Distribution

Having a certain number of artifacts, the generator spreads these across the map. Distribution methods don't have much overall **structure** (B), but can be sophisticated with the odds of picking each option (A). Some **randomness** prevents the generator from picking the same option twice (or twice in a row).

Conditional rules (A2) provide greater depth on the distribution but require more complexity. Having determined parameters that can be set for each option and for each conditional option, the generator can choose to compare the fixed parameters.

An **example**:

While wolves can be found in every part of every maps (A), grizzlies can only be found near the mountains (A2)

Tile-based

Make modular, identically-sized slots, and create **hand-made** solutions that will fill these slots. The arrangement of these slots, chosen randomly or by different pre-ordered sets, will create the artifacts needed.

The [tile-based](#) artifacts have a structure (B) that can still provide different gameplay every time they are arranged. Internally, every slot has no **flexibility**, but can be achieved through the displacement of numerous tiles (B2).

Instead of hand-made tiles they can be arranged by the same generator providing it with some parameters. These parameters can be hard to manage and tune, depending on the number of different tiles and the internal difference of each tile.

It works for problems that can be broken in pieces where **internal structure matters**, but can still create interesting behaviours when combined in **different orders**.

An example:

Settlers of Catan: the tiles are pre-made, but every game is played in a different way due to the different placement of the tiles.



Plot and Parcels - XCOM 2

The developers of XCOM2 started working on the procedural maps differentiating the various levels based on the “**biome**” of the map, whether it be city, wilderness or an alien facility.

Made holes on the landscape where they wanted to place a building, and created the buildings to put in each spot on another map, and then stream-in them in the master map (B).

The master layout map called **Plot**, the streamed-in ones called **Parcels**.

Every plot needed to have **specific rules** to be used in as many missions as possible.

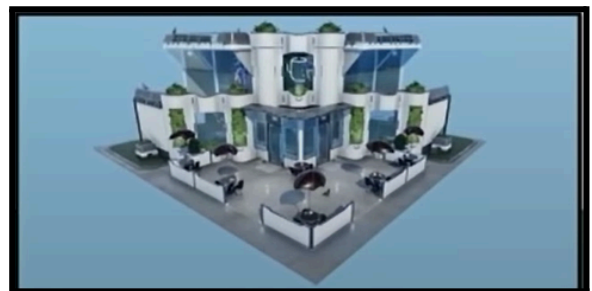


Initial Direction

- Cut holes in static map for buildings
- Stream buildings in from separate maps
- Plot - Master layout map
- Parcels – Streamed-in buildings

The parcels also had specific parameters and rules:

- Three buckets of parcels
 - Small - 16x16
 - Medium - 16x32
 - Large - 32x32
- Plots had spaces for each size
- Parcels **subdivide**
- Have a **random facing**



In addition, to differentiate maps and the gameplay, were introduced **Plot Cover Parcels (PCP)**. They had different rules than Parcels.

- It's ok to use it **multiple times**
- **Small and generic** enough. Unless two of the same type, facing the same direction, were placed one near the other, players couldn't tell the same asset were utilized multiple times

Parcels and PCP needed guidelines, to avoid the problem of having a store with the entrance facing away from the road. This **eliminated some randomness** to give **credibility** to the assets.

It was also achieved by **zoning** the maps. An example is to divide a city into inhabited centers and shopping streets. A house made more sense when placed near other houses, then between two shopping centers (B2).

Identity and distribution

Maps

Creating a large number of maps can be challenging, particularly if the intent of the game is to always provide a new experience for the player.

Learning a map means that the player will know how to face it every time he finds himself in it. In addition, even if a map is different, it can be perceived as the same as another map.

Changing some artifacts (such as resources, obstacles or enemies) might not be enough to give the **feeling** of a totally **different** map.

One way to help solve this problem is by completely changing the starting point of the player.

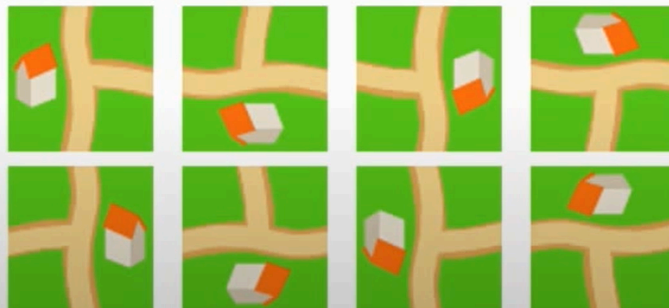


In this example from [XCOM2 War of the Chosen](#), the map is shared between two levels. They manage to completely change the feeling of it and how it plays by placing the player start on the **other side of the map** and passing **from day to night**.

Another way is **reflecting or rotating** the map, or parts of it, **on the axes**. In essence the map doesn't change, and the assets remain the same, but the level can be perceived in a different way, enough to differentiate this new map from the previous non-rotated one

Module transforms

- Rotation about z-axis
- Reflection about x- or y-axis



[Procedural Level Design in Eldritch](#)

It may be challenging to create a huge number of maps and hoping to find the ones perfect for the game. Testing every map of the hundreds created can be time consuming. This is why you might want to **set some parameters** for the generator, or **set them by hand**.

Some useful parameters can be the starting point, the end point or **landmarks**.

Objects

Another way of differentiating the maps is with the shape, color and placement of the objects. Has been discussed before how generic and small objects can be repeated without almost an issue.

Randomizing their **materials and textures** can also be another layer of differentiation.



Instead of having multiple objects, with similar shapes and different texture, the same object can be placed multiple times. It is something that might take a lot of time if it is made by hand.

A possible solution is to create every object and every map with the same style of texture (a forest biome) and, **when the game starts**, have an **algorithm** change the materials and texture of determined objects (snow biome).

In this way, the same map can be perceived and played in very different ways.



Of course **not everything is replaceable**. A chunk of ice cannot be simply changed with some moss or dust on it.



Landmarks are objects of such interest and importance that their position can hardly be randomized by a generator. Parameters can be set, but there is the risk of creating such difficult parameters to fulfill that only a few landmarks can be placed by the generator.

Hades does something similar: every room is handmade, and then are randomly chosen for a run.

Examples in other games

Moon Hunters

The **world map** is composed of numerous tiles, and every tile is composed of different encounters (combat, dialogue etc). Every tile can have **two destinations**, and the editor finds the destinations and **draws paths** between them.



The encounter maps themselves are built with **nodes**. Chooses a beginning and an end point and creates the path between those two points. Then makes deviations and **side paths**. The nodes are hand-built and there are over 100 of them.

Hades

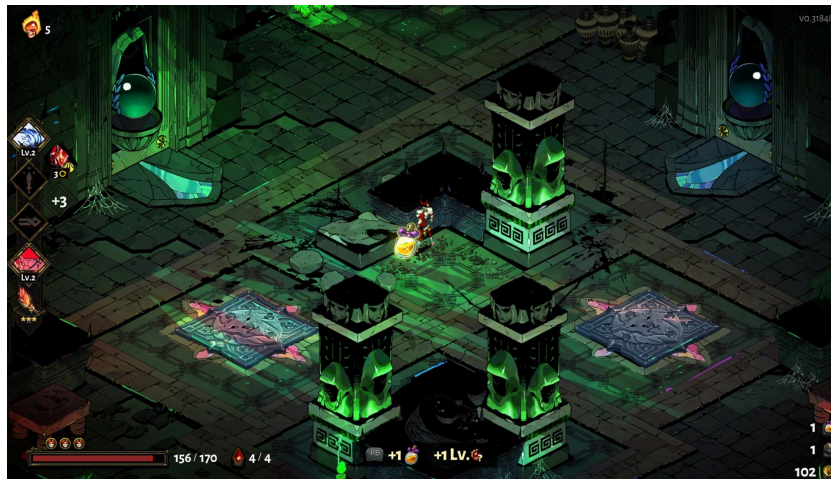
It's already been said that the rooms of Hades are not procedurally generated but hand-crafted. Their presence and position during the run is randomized, with certain parameters that need to be fulfilled:

- The room before a boss must be an empty room (with a boon/loot) or a shop
- Smaller rooms tend to appear **early** on in biomes
- Larger rooms tend to appear **later** on in biomes

The difference between the four biomes is clear and strictly connected to the **mood** of the biome and the goals of the **level design**.

In the Asphodel the rooms are a "series of archipelagos surrounded by lava" and are created to "feel horizontal", the Barge of Death being the main example.

On the other hand, the rooms in Tartarus "we wanted medium-sized chambers that are almost always completely walled in, making it forgiving for players learning about the Cast while providing extra Wall Slam damage".



Don't Starve

The game world is generated and with it its content. The flow of how the world generation goes is:

1. For a given **Preset**, **Tasks** are selected, some mandatory and few optional
 - a. Each world generation preset has different configuration that causes different contents
2. From each Task in the Preset, a **graph** of Task Nodes is created
 - a. Tasks are written-like story elements - overall goals for what the world should have. Each task specifies locks, keys, rooms and a background [turf](#) which determines what entities are **not** allowed there.
3. Each task generates a set of **Rooms** and then linked together
 - a. Rooms are small biomes that contain certain elements. When a room is generated, a set of **nodes** are created as well, spaced out from each other to ensure that objects in the room are distributed relatively evenly.
4. Tasks linked together based on **Locks** and **Keys**. Task that don't get a Key for their Lock get discarded
 - a. Locks and Keys specifies how tasks can be connected to each other.
 - b. Each Key can unlock one or more Locks
 - c. Tasks added later are usually situated at the edges of the world, and these will usually require keys gathered at the center of the world
5. Background rooms are added to each task
6. Set pieces (both mandatory and optional) and teleported pieces get assigned to rooms
7. The world generate a spatial arrangements from the graph and **converts it to tiles**
8. Fixes island in caves, and checks that rooms have enough space (restarting if not)
9. Entities are added at each of the nodes, and roads are drawn, avoiding certain rooms

[Detailed generation description](#)

Conclusions

World generators must be **well-studied** before their creation and are **strictly linked to the needs** of the developers and the game. So the first step is **knowing what you want** from the generator, setting some parameters knowing that not all that is created will be good.

The methods for creating such generators are, once again, linked to what your goal is. More **randomness** can be achieved with encapsulated knowledge of options and conditional rules, and distribute these on the map.

More structure can be provided, and for various **reasons** (lightning, shadows, landmarks) these tiles should be **hand-crafted** and then shuffled with the generator.

One valid option is to follow the **Plot** and **Parcels** method used in XCOM2. Create a **landscape and some fixed elements**. Huge elements can be placed by the generator if they all need to have a strong identity. On the other hand, small objects can be repetitive.

The main goal, beside not having all assets look the same, is to provide an always **different experience of gameplay** for replayability, surprise and desire to start new runs.

The tricks to obtain this feeling are many, and often concern the rotation of maps/objects, different lightning, different spawn points and the possibility to **change materials and texture** of objects and landscape.